

Gemma 4 (E4B) Fine-Tuning Notebook'u — Nasıl Çalıştırılır?

Antalya LLM Tabanlı Uygulama Geliştirme Eğitimi • Dr. Murat Altun

Bu rehber, [gemma4_json_finetune.ipynb](#) dosyasını **ücretsiz** bir GPU üzerinde uçtan uca çalıştırmayı adım adım anlatır. Kod bilmenize gerek yok — hücreleri sırayla çalıştırmanız yeterli.

Notebook ne yapıyor? Dağınık bir belgeyi (fatura, tıbbi kayıt, sipariş...) + bir JSON şemasını alıp, şemaya birebir uyan **geçerli JSON** üreten bir model eğitir. Küçük ve açık bir model olan **Gemma 4 E4B**'yi, **QLoRA** yöntemiyle tek ücretsiz GPU'da fine-tune ederiz.

0. Önce kısa kavramlar (LoRA / QLoRA nedir?)

Notebook'un içinde bunlar analogilerle detaylı anlatılıyor; burada bir cümlelik hatırlatma:

Kavram	Bir cümlede
Base vs Instruct (-it)	Base model ham cümle tamamlayıcıdır; Instruct (bizim kullandığımız -it) komut anlar, cevap verir.
Quantization (4-bit)	Modelin 16-bit hali ~10 GB (sığmaz); 4-bit 'e sıkıştırınca ~6 GB olur → ücretsiz GPU'ya sığar, doğruluk neredeyse aynı.
LoRA	Donuk modeli baştan yazmak yerine kenara küçük " not kağıtları " (eğitilebilir matrisler) iliştiririz. Model aynı kalır, notlar görevi öğrenir.
QLoRA	LoRA + 4-bit taban. Sadece parametrenin ~%0.5'ini eğitiriz → ucuz, hızlı, ücretsiz GPU'da çalışır.
SFT	<i>Supervised Fine-Tuning</i> — modele "bu girdiye şu çıktı" örneklerini göstererek öğretmek.

A. Kaggle'da Çalıştırma (önerilen — en stabil ücretsiz GPU)

A.1. Hesap ve GPU hakkı

1. kaggle.com adresine gir, ücretsiz hesap aç (Google ile giriş yapılabilir).
2. **Telefon doğrulaması yap** (Settings → Phone Verification). Bu olmadan GPU açılmaz.
3. Doğrulama sonrası haftalık **~30 saat ücretsiz GPU** hakkın olur.

A.2. Notebook'u yükle

1. Üstten **Create → New Notebook**.
2. Açılan notebook'ta **File → Import Notebook**.
3. [gemma4_json_finetune.ipynb](#) dosyasını sürükle-bırak / seç.

A.3. GPU ve İnterneti aç (KRİTİK)

Sağ paneldeki **Session options** (: / Settings) bölümünden: - **Accelerator** → **GPU T4 x2** (veya GPU P100) - **Internet** → **On** (model ve veri seti indirilecek; kapalıysa çalışmaz!) - **Persistence** → gerekmez (varsayılan kalabilir)

A.4. Çalıştır

- Üstten **Run All** (veya her hücreyi sırayla ►).
- **İlk 2-4 dakika**: Unsloth kurulumu + Gemma 4 modelinin (~6 GB) indirilmesi. Sabırlı ol, normaldir.
- **Sonraki ~10-15 dakika**: Eğitim (`trainer.train()`). Loss (kayıp) değerinin düşmesini izle → model öğreniyor demektir.

A.5. Sonucu gör

- **Baseline** hücresi: fine-tune *öncesi* geçerli-JSON oranı (düşük çıkar, model gevezelik yapar).
- **Sonrası** hücresi: fine-tune *sonrası* oran (yüksek çıkar, tertemiz JSON).
- Aradaki fark = dersin kanıtı.

A.6. Modeli indir (opsiyonel)

- Eğitim sonrası [gemma4-json-lora/](#) klasörü oluşur → sağdaki **Output** sekmesinden indirebilir veya Kaggle'da "Model" olarak yayınlatabilirsin.
-

B. Google Colab'da Çalıştırma (alternatif)

1. colab.research.google.com → File → Upload notebook → `gemma4_json_finetune.ipynb`.
2. Runtime → Change runtime type → T4 GPU seç, kaydet.
3. Runtime → Run all.
4. Colab ücretsiz kotası Kaggle'dan daha değişkendir; GPU bulunamazsa birkaç saat sonra tekrar dene veya Kaggle'ı tercih et.

Notebook, Colab ile Kaggle'ı **otomatik ayırt eder** (kurulum hücresi kendini ayarlar) — sen sadece Run All demen yeterli.

C. Sık Karşılaşılan Hatalar ve Çözümleri

Belirti	Sebeup	Çözüm
<code>No CUDA GPUs are available</code>	GPU seçilmemiş	Accelerator = GPU T4 yap, oturumu yeniden başlat
İndirme takılıyor / <code>ConnectionError</code>	İnternet kapalı	Internet = On yap
<code>gated repo</code> / 401 model hatası	Model erişimi	Kaggle → Add-ons → Secrets → <code>HF_TOKEN</code> ekle; notebook notundaki <code>login(...)</code> satırını çalıştır
<code>CUDA out of memory</code>	Bellek doldu	<code>max_seq_length</code> 'i 2048 → 1024 düşür veya oturumu Restart edip baştan çalıştır
Kurulum çok uzun / donmuş	İlk kurulum + model indirme	Normal (2-4 dk). En az 5 dk bekle
<code>NameError: model</code>	Hücreler sırasız çalıştı	Baştan Run All yap (yukarıdan aşağı sırayla)
Eğitim çok yavaş	Büyük batch / uzun seq	<code>per_device_train_batch_size=1</code> kalsın; <code>max_steps</code> demoda 60

D. Çıktıyı Nasıl Yorumlarız?

1. **Loss (kayıp)** eğitim boyunca **düşüyorsa** model JSON kalıbını öğreniyordur.
2. **Baseline oranı** (ör. %60) düşük, **fine-tuned oranı** (ör. %95+) yüksek olmalı — bu *iyileşmenin objektif kanıtıdır*.
3. Örnek çıktıda `json.loads()` **hatasız** çalışıyorsa → çıktı gerçekten geçerli JSON'dur, doğrudan koda/veritabanına akar.

Bilimsel disiplin: "Sanırım iyi oldu" demiyoruz. Öncesini ölçüyoruz, sonrasını ölçüyoruz, aynı test setinde kıyaslıyoruz. Test seti eğitimden ayırdır (data leakage yok).

E. Sonraki Adımlar (öğrenci için)

- `max_steps=60` yerine `num_train_epochs=2` ile **tam eğitim** yapıp oranı tekrar ölç.
- `jsonschema.validate` ile sadece "parse oldu mu"yu değil, **şemaya tam uygunluğu** ölç.
- **Kendi Türkçe verinle** (fatura/dilekçe → JSON) aynı hattı çalıştır — gerçek dünyada asıl beceri budur.
- Modeli **GGUF**'a çevirip **Ollama** ile yerelde çalıştır (offline, ücretsiz).

Kaynak veri seti: `paralog/json_data_extraction` (Hugging Face, apache-2.0) — notebook çalışırken otomatik iner.